Introduction to Processor Architecture

Pipelined RISC-V Processor – Phase II Report

Team 14 - ARMed Civilians

Yogesh Krishna	2024102035
Pidapa Haarthiv	2023102035
Shruti Dhasmana	2022102081

March 10, 2026

Contents

1 Overview **3**
1.1 Changes from the Sequential Design . . . 3
1.2 Naming Convention for Pipeline Signals . . . 4

2 Pipeline Registers **4**
2.1 IF/ID Register . . . 4
2.2 ID/EX Register . . . 5
2.3 EX/MEM Register . . . 5
2.4 MEM/WB Register . . . 6

3 Data Forwarding **6**
3.1 Forwarding Unit (`ForwardingUnit.v`) . . . 6
3.1.1 Forwarding Selection . . . 6
3.1.2 Store-Data Forwarding (Bonus) . . . 6
3.2 Branch Forwarding Unit (`BranchForwardingUnit.v`) . . . 7

4 Hazard Detection Unit (`HazardDetectionUnit.v`) **7**
4.1 Stall Cases . . . 7
4.1.1 Case 1: Load-Use Hazard (non-branch) . . . 7
4.1.2 Case 2: Branch N-1 Dependency . . . 7
4.1.3 Case 3: Branch N-2 Load Dependency . . . 8

5 Control Hazard Handling **8**
5.1 Branch Resolution in the ID Stage (Bonus) . . . 8

6 Test Cases from the Instruction Program **8**
6.1 Simultaneous EX and MEM Forwarding . . . 8
6.2 Write-Before-Read Bypass (Dependency from (N-3)th cycle) . . . 9
6.3 EX-to-EX Forwarding (Dependency from (N-1)th cycle) . . . 9
6.4 MEM-to-EX Forwarding (Dependency from (N-2)th cycle) . . . 9
6.5 Branch Taken with No Data Dependency (1-cycle flush penalty) . . . 9
6.6 Branch Not Taken (No Flush) . . . 10

7 Waveforms **10**
7.1 From given instructions_exp.txt . . . 10
7.2 From custom instructions for checking . . . 11

8 Results **12**
8.1 Functional Correctness . . . 12
8.2 Cycle Count . . . 12
8.3 Hazard Scenarios Verified . . . 13

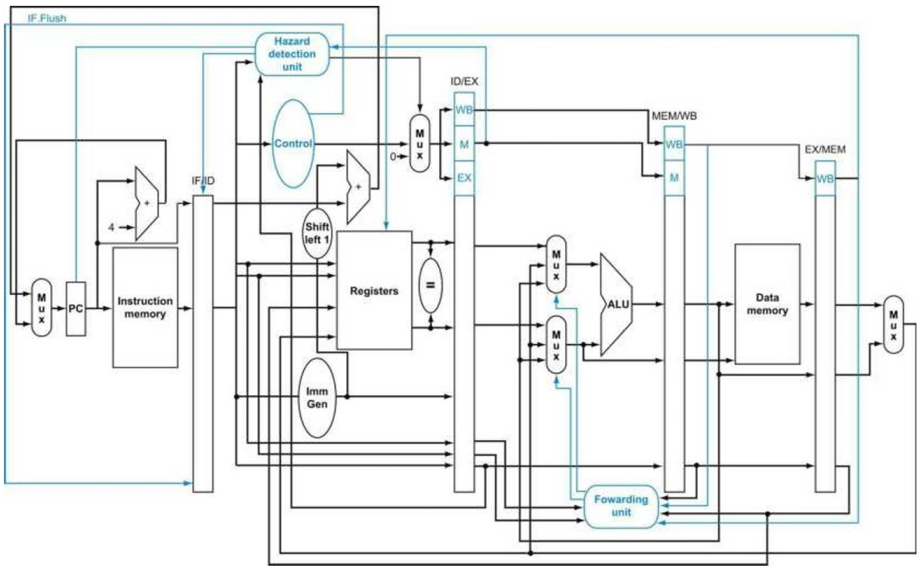
9 BONUS sections **13**

10 Files specific to Pipelined Implementation **14**

11 Team Contributions **14**

1 Overview

This report covers the additions made on top of the sequential RISC-V processor to convert it into a 5-stage pipelined processor. The base datapath blocks (ALU, Control Unit, Register File, Data Memory, Instruction Memory, Immediate Generator, ALU Control, Adder, ShiftLeft1, MUX) remain functionally identical to the sequential design.



1.1 Changes from the Sequential Design

The following were added or modified:

1. **Four sets of pipeline registers** between the five stages (IF/ID, ID/EX, EX/MEM, MEM/WB).
2. **Forwarding Unit** – resolves data hazards by bypassing results from later stages back to the EX stage ALU inputs.
3. **Hazard Detection Unit** – detects load-use and branch-related hazards that cannot be resolved by forwarding alone; stalls the pipeline by inserting bubbles.
4. **Branch resolution moved to ID stage** – a dedicated `equalityComparator` resolves `beq` in the Decode stage (rather than the EX stage), reducing the branch penalty from 2 flush cycles to 1.
5. **Branch Forwarding Unit** – provides forwarding paths to the branch comparator so that the ID-stage comparison uses the latest values.
6. **Store-data forwarding (bonus)** – a special forwarding path for store instructions whose data register depends on a value being written back simultaneously.
7. **4-to-1 MUX** (`mux4x2.v`) – new multiplexer module used for forwarding selection (4 instances).
8. **PC enable signal** – the PC module now has an `enable` input (`PCWrite`) that can freeze the PC during stalls.

1.2 Naming Convention for Pipeline Signals

All pipeline register signals follow a prefix convention that makes them traceable in the codebase:

- `IFID*` – internal signals of the IF/ID pipeline register (e.g., `IFIDIR`, `IFIDPC`).
- `IDEX*` – internal signals of the ID/EX register (e.g., `IDEXrs1`, `IDEXALUOp`, `IDEXrd`).
- `EXMEM*` – internal signals of the EX/MEM register (e.g., `EXMEMALUResult`, `EXMEMrd`).
- `MEMWB*` – internal signals of the MEM/WB register (e.g., `MEMWBReadData`, `MEMWBrd`).
- `BMIDEX*` – "Bubble Mux" outputs that feed into ID/EX. When a bubble is asserted, these are zeroed to insert a NOP (e.g., `BMIDEXRegWrite`).
- Control signals `Forward*` come from forwarding units; hazard signals `PCWrite`, `IFIDEnable`, `Bubble` come from the Hazard Detection Unit.

2 Pipeline Registers

All four pipeline registers live inside `processor.v` as `reg` declarations and update on every rising clock edge. On reset, the instruction-holding registers (`IFIDIR` and `IDEXIR`) are loaded with the NOP encoding `0x00000013` (which is just `addi x0, x0, 0`), and all write-enable and memory-enable signals are cleared so no accidental writes happen during startup.

2.1 IF/ID Register

Signal	Width	Description
`IFIDIR`	32	Fetched instruction. Used inside the ID stage as `Instruction` via a continuous assign.
`IFIDPC`	64	PC of the fetched instruction (used for branch target computation).

Special behavior:

- When `IFIDEnable = 0` (stall), both fields hold their previous values.
- When `IFFlush = 1` (taken branch), `IFIDIR` is overwritten with the NOP instruction to nullify the wrongly fetched instruction.

2.2 ID/EX Register

Signal	Width	Source (from ID stage)
`IDEXALUOp`	2	ALU operation specifying signal
`IDEXALUSrc`	1	select signal for ALU input B
`IDEXMemRead`	1	read enable for Memory
`IDEXMemWrite`	1	write enable for Memory
`IDEXMemtoReg`	1	Source register address for Write-back
`IDEXRegWrite`	1	Register write enable
`IDEXrs1`	64	Read data from register `rs1`
`IDEXrs2`	64	Read data from register `rs2`
`IDEXImm`	64	Sign-extended immediate
`IDEXIR`	32	Full instruction (for funct3/funct7)
`IDEXrd`	5	Destination register address
`IDEXrs1Addr`	5	Source register 1 address
`IDEXrs2Addr`	5	Source register 2 address

Note: The six control signals pass through the **Bubble Mux** before being latched. When `Bubble = 1`, all six are zeroed, effectively converting the instruction in ID/EX into a NOP. The data fields (`IDEXrs1`, `IDEXrs2`, `IDEXImm`, etc.) are always written regardless of bubble, since a NOP with zero control signals has no side effects.

2.3 EX/MEM Register

Signal	Width	Source (from EX stage)
`EXMEMALUResult`	64	ALU output
`EXMEMrs2`	64	Forwarded rs2 value (for store data), sourced from `ALUm2Op`, (it's the latest value of rs2 for EX stage, so we use it)
`EXMEMrs2Addr`	5	rs2 register address (for SD forwarding)
`EXMEMrd`	5	Destination register
`EXMEMMemRead`	1	Memory read enable
`EXMEMMemWrite`	1	Memory write enable
`EXMEMRegWrite`	1	Register write enable
`EXMEMMemtoReg`	1	Write-back source

Note: `EXMEMrs2` stores the forwarded rs2 value (output of the forwarding mux, `ALUm2Op`) rather than the raw `IDEXrs2`. This ensures that if rs2 was forwarded in the EX stage, the correct value reaches the Data Memory during the MEM stage.

2.4 MEM/WB Register

Signal	Width	Source (from MEM stage)
`MEMWBReadData`	64	Data memory read output
`MEMWBALUResult`	64	ALU result passed through
`MEMWBrd`	5	Destination register
`MEMWBRegWrite`	1	Register write enable
`MEMWBMemtoReg`	1	Write-back source (0 = ALU, 1 = memory)

3 Data Forwarding

3.1 Forwarding Unit (`ForwardingUnit.v`)

The Forwarding Unit detects when an instruction in the EX stage needs a result that is still in the EX/MEM or MEM/WB pipeline register. It produces three signals:

- `ForwardA` [1:0] – selects ALU operand A source.
- `ForwardB` [1:0] – selects ALU operand B source.
- `Forward_sd` [1 bit] – selects store-data source.

Inputs: `EXMEMRegWrite`, `MEMWBRegWrite`, `EXMEMrd`, `MEMWBrd`, `EXMEMrs2Addr`, `IDEXrs1Addr`, `IDEXrs2Addr`.

3.1.1 Forwarding Selection

`ForwardA/B`	MUX input	Condition (A uses rs1Addr, B uses rs2Addr)
`2'b00`	`IDEXrs1/2`	No hazard
`2'b10`	`EXMEMALU`	EX hazard: `EXMEMRegWrite && EXMEMrd` $\neq$ `0 && EXMEMrd == src`
`2'b01`	`write_back`	MEM hazard: `MEMWBRegWrite && MEMWBrd` $\neq$ `0 && MEMWBrd == src`

EX hazard has higher priority over MEM hazard (checked first via `if-else`).

The forwarded values are routed through two `mux4x2` instances (`ALUm1` for operand A, `ALUm2` for operand B) before entering the ALU.

3.1.2 Store-Data Forwarding (Bonus)

When a store instruction reaches the MEM stage and its data register (`EXMEMrs2Addr`) matches the destination of the instruction currently in WB, the store data must be forwarded:

`Forward_sd = 1` when `MEMWBRegWrite && MEMWBrd` $\neq$ `0 && MEMWBrd == EXMEMrs2Addr`.

A `mux` is used to (`sdMUX`) select between `EXMEMrs2` (normal) and `write_back` (forwarded) based on `Forward_sd`. The output (`sdMUXOp`) feeds into the Data Memory's write port.

3.2 Branch Forwarding Unit (`BranchForwardingUnit.v`)

Since the branch condition is resolved in the **ID stage** (not EX), and the branch comparator needs the latest register values, a separate forwarding unit is needed.

Inputs: `EXMEMRegWrite`, `MEMWBRegWrite`, `EXMEMrd`, `MEMWBrd`, `IFIDrs1` (= `Instruction[19:15]`), `IFIDrs2` (= `Instruction[24:20]`).

Outputs: `ForwardCompA` [1:0], `ForwardCompB` [1:0] – same encoding as `ForwardA/B`.
Two `mux4x2` instances (`compMuxA`, `compMuxB`) feed the forwarded values to the `equalityComparator`.

4 Hazard Detection Unit (`HazardDetectionUnit.v`)

The Hazard Detection Unit detects data and control hazards that *cannot* be resolved by forwarding alone. When a stall is required, it asserts:

- `PCWrite = 0` – freezes the PC.
- `IFIDEnable = 0` – freezes the IF/ID register.
- `Bubble = 1` – zeroes the control signals entering ID/EX (via the Bubble Mux), inserting a NOP.

Inputs: `IDEXMemRead`, `IDEXRegWrite`, `EXMEMMemRead`, `EXMEMrd`, `Instruction` (current IF/ID instruction), `IDEXIR` (ID/EX instruction).

The HDU internally derives:

```
wire [4:0] IDEXrd = IDEXIR[11:7];
wire [4:0] rs1 = Instruction[19:15];
wire [4:0] rs2 = Instruction[24:20];
wire isBEQ = (Instruction[6:0] == 7'b1100011);
```

4.1 Stall Cases

4.1.1 Case 1: Load-Use Hazard (non-branch)

When a `ld` instruction is in ID/EX and the next instruction in IF/ID reads the loaded register, forwarding cannot help because the data is not available until the end of MEM.

For I-type (ld, addi): only `rs1` is a source register.
`Stall if: IDEXMemRead && IDEXrd` $\neq$ `0 && IDEXrd == rs1`

For R-type / sd: both `rs1` and `rs2` are source registers.
`Stall if: IDEXMemRead && IDEXrd` $\neq$ `0 && (IDEXrd == rs1 || IDEXrd == rs2)`

4.1.2 Case 2: Branch N-1 Dependency

When `beq` is in IF/ID and the *immediately preceding* instruction (in ID/EX) writes to a register that `beq` reads. Because the branch comparator is in the ID stage, it needs the value one cycle before forwarding from EX/MEM is available.

`Stall if: isBEQ && IDEXRegWrite && IDEXrd` $\neq$ `0 && (IDEXrd == rs1 || IDEXrd == rs2)`

- If the preceding instruction is an ALU instruction: 1 stall, then forward from EX/MEM.

- If the preceding instruction is a `ld`: 1st stall here; after the `ld` moves to EX/MEM, Case 3 triggers a 2nd stall.

4.1.3 Case 3: Branch N-2 Load Dependency

When `beq` is in IF/ID and a load instruction two positions earlier (now in EX/MEM) writes to a register that `beq` reads. The loaded data only becomes available at the end of MEM, so one more stall is needed before it can be forwarded from MEM/WB.

`Stall if: isBEQ && EXMEMMemRead && EXMEMrd` $\neq$ `0 && (EXMEMrd == rs1 || EXMEMrd == rs2)`

5 Control Hazard Handling

5.1 Branch Resolution in the ID Stage (Bonus)

Instead of resolving branches in the EX stage (which would require flushing 2 instructions), we resolve branches in the **ID stage** using a dedicated `equalityComparator`:

```
equalityComparator comp1(.a(compA), .b(compB), .equal(rs1EqualRs2));
```

The inputs `compA` and `compB` come from the Branch Forwarding MUXes (`compMuxA`, `compMuxB`), which provide the most current (possibly forwarded) register values.

The branch decision: `IFFlush = Branch & rs1EqualRs2`.

- When `IFFlush = 1`: the IF/ID register is loaded with NOP (`0x00000013`), and the PC is redirected to the branch target (`pcPlusImm = IFIDPC + ShiftLeft1(immediate)`).
- Only 1 instruction in the IF stage needs to be flushed, giving a **1-cycle branch penalty** (vs. 2 cycles if resolved in EX).

6 Test Cases from the Instruction Program

The test program (`instructions_exp.txt`) given had many key hazard scenarios implemented. Below we describe each scenario with the relevant instruction sequence and how the pipeline handles it.

6.1 Simultaneous EX and MEM Forwarding

```
addi x2, x0, 5        // writes x2
addi x3, x0, 10       // writes x3
add  x1, x2, x3       // reads x2 (MEM fwd) and x3 (EX fwd)
```

When `add x1, x2, x3` enters the EX stage, `addi x3` is in the EX/MEM register (1 cycle ahead, rd = x3) and `addi x2` is in the MEM/WB register (2 cycles ahead, rd = x2). The Forwarding Unit sets ForwardB = 2'b10 (EX hazard, forwarding x3 from EXMEMALUResult) and ForwardA = 2'b01 (MEM hazard, forwarding x2 from the write_back value). Both operands are forwarded simultaneously with no stall needed.

6.2 Write-Before-Read Bypass (Dependency from (N-3)th cycle)

```
addi x2, x0, 5        // writes x2 (cycle N)
addi x3, x0, 10       // (cycle N+1)
add  x1, x2, x3       // (cycle N+2)
sub  x2, x2, x3       // reads x2 (cycle N+3, WB and ID same cycle)
```

When `sub x2, x2, x3` enters the ID stage, `addi x2` is simultaneously in the WB stage writing x2 to the register file. Neither the EX/MEM nor MEM/WB forwarding paths cover this case. Instead, the Register File's combinational write-before-read bypass detects that the write destination matches the read source and routes `write_data` directly to `read_data1`, providing the correct value of x2 = 5 without a stall.

6.3 EX-to-EX Forwarding (Dependency from (N-1)th cycle)

```
and  x4, x3, x3       // writes x4 = 10
and  x5, x3, x4       // reads x4 (EX hazard: forward EXMEMALUResult)
```

When `and x5, x3, x4` reaches the EX stage, `and x4` is in the EX/MEM register with rd = x4. The Forwarding Unit detects `EXMEMRegWrite && EXMEMrd(x4) == IDEXrs2Addr(x4)` and sets ForwardB = 2'b10, routing the ALU result of `and x4` directly to the ALU's operand B with no stall.

6.4 MEM-to-EX Forwarding (Dependency from (N-2)th cycle)

```
and  x4, x3, x3       // writes x4 (cycle N)
and  x5, x3, x4       // writes x5 (cycle N+1)
or   x6, x2, x4       // reads x4 (cycle N+2, MEM hazard)
```

When `or x6, x2, x4` reaches the EX stage, `and x4` is in MEM/WB (2 cycles ahead, rd = x4) while `and x5` is in EX/MEM (rd = x5, no match). The Forwarding Unit sets ForwardB = 2'b01, forwarding x4's value from the MEM/WB write-back path. No stall is needed.

6.5 Branch Taken with No Data Dependency (1-cycle flush penalty)

```
ld   x11, 24(x5)      // (no dependency on branch operands)
beq  x4, x5, 8        // x4 == x5 == 10 -> branch TAKEN
beq  x0, x0, 0        // FLUSHED (replaced with NOP)
add  x13, x1, x10     // branch target, executes normally
```

When `beq x4, x5, 8` enters the ID stage, both x4 and x5 were written many cycles earlier (by `and x4` and `and x5`), so their values are available directly from the register file with no forwarding needed. The equality comparator determines rs1EqualRs2 = 1 in the same cycle. Since Branch = 1, IFFlush = Branch & rs1EqualRs2 = 1, causing the instruction in IF (`beq x0, x0, 0`, the infinite loop) to be replaced with a NOP. The PC is redirected to PC + 8 = 56, fetching `add x13, x1, x10` in the next cycle. The branch penalty is exactly **1 cycle** (the flushed instruction slot).

6.6 Branch Not Taken (No Flush)

If `rs1` $\neq$ `rs2`, the branch is not taken: `IFFlush = 0`, and the instruction in IF proceeds normally with no penalty.

7 Waveforms

7.1 From given instructions_exp.txt

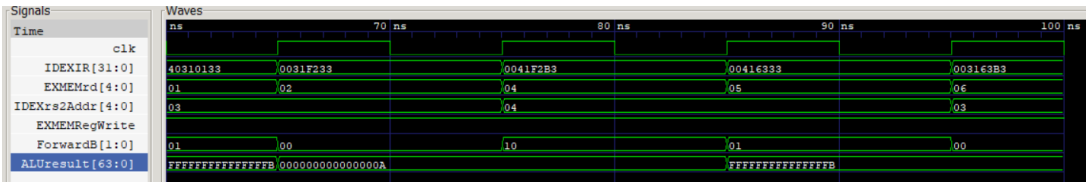


Figure 1: At 75ns (cycle 8): ForwardB=10, EXMEMrd=x4, ALUresult=0xA, A case of EX/MEM to EX forwarding

`Note:` Clock Cycle posedge time instant = 5ns(for reset) + (Clock cycle number - 1)$\times 10ns$

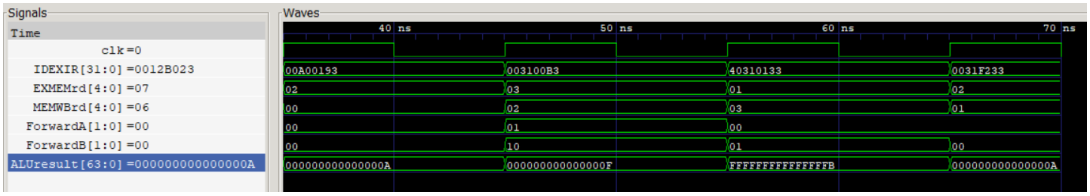


Figure 3: At 45ns (cycle 5): ForwardA=01, ForwardB=10, ALUresult=0xF. A case of simultaneous EX/MEM to EX forwarding (x3 via ForwardB) and MEM/WB to EX forwarding (x2 via ForwardA)

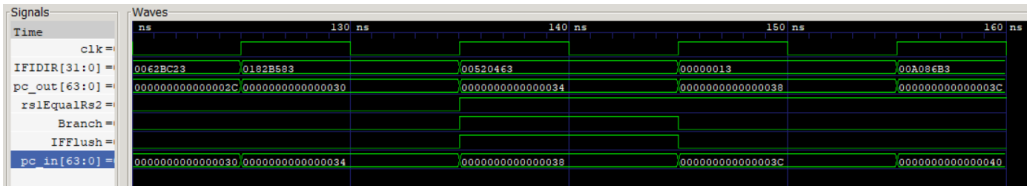


Figure 4: At 135 ns (Cycle 14): Branch detection ($rs1EqualRs2 = 1$) triggers a pipeline flush ($IFFlush = 1$) as `beq x4, x5, 8` is resolved in the ID stage. By 145 ns (Cycle 15), a NOP (0x00000013) is inserted into the IF/ID register and the PC jumps to the target instruction.

7.2 From custom instructions for checking

```
addi x11, x0, 17

ld   x24, 121(x11)     # Load x24 from memory
sub  x25, x0, x24      # Dependent instruction: uses x24 immediately

add  x0, x0, x0
add  x0, x0, x0
add  x0, x0, x0
add  x0, x0, x0
```

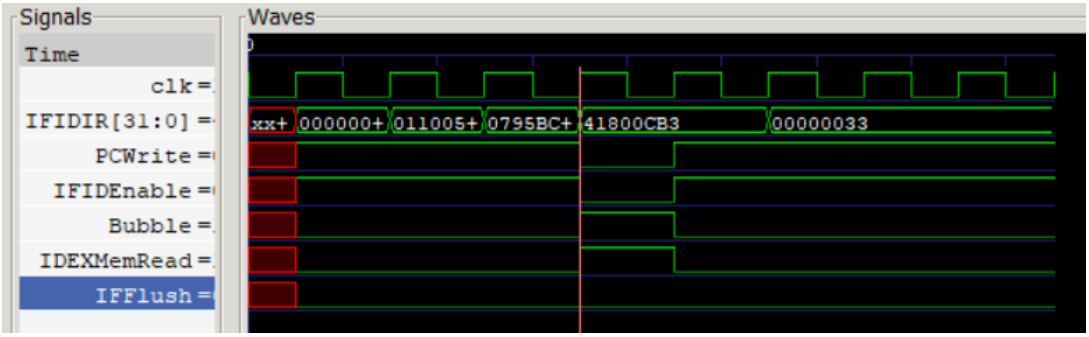


Figure 5: At 35 ns (cycle 4), a load–use data hazard occurs: `ld x24, 121(x11)` is in the EX stage with IDEXMemRead = 1, while the following `sub x25, x0, x24` in the ID stage requires x24. The Hazard Detection Unit stalls the pipeline for one cycle (PCWrite = 0, IFIDEnable = 0, Bubble = 1), keeping `sub` in ID and inserting a NOP into EX. The stall is released at 45 ns; once `ld` reaches MEM/WB at 55 ns, the loaded value is forwarded to `sub` in EX via the MEM/WB forwarding path.

```
addi x5, x0, 63      # x5 = 63
add  x5, x5, x5      # x5 = 126
addi x8, x0, 252   # x8 = 252 (Initialize x8 to match x13 for the beq)

add  x13, x5, x5   # x13 = 252. ALU result not ready for BEQ in ID stage
beq  x13, x8, 8    # Branch if x13 == x8. Requires x13 from prev. instr.
addi x31, x0, -1     # This instruction is fetched but FLUSHED
add  x14, x13, x13   # Target of branch (offset 8 from instr. after beq)

add  x0, x0, x0
add  x0, x0, x0
add  x0, x0, x0
add  x0, x0, x0
```

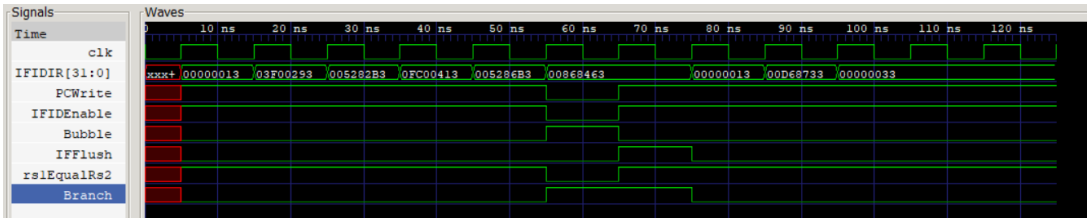


Figure 6: At 55 ns (cycle 6), a branch data hazard occurs: `add x13, x5, x5` is in the EX stage and the following `beq x13, x8, 8` in the ID stage requires x13 for comparison. The Hazard Detection Unit stalls the pipeline for one cycle (PCWrite = 0, IFIDEnable = 0, Bubble = 1), keeping `beq` in the ID stage and holding `addi x31, x0, -1` frozen in IF. At 65 ns (cycle 7), the stall is released and `add x13`'s result (252) is in EX/MEM; the Branch Forwarding Unit forwards it to the comparator (ForwardCompA = 10). The equality comparator evaluates `rs1EqualRs2 = 1`, asserting IFFlush = 1, which replaces the wrongly-fetched `addi x31` with a NOP at the 75 ns posedge. The pipeline resumes at 75 ns (cycle 8), fetching the branch target `add x14, x13, x13`.

8 Results

8.1 Functional Correctness

The processor was verified against the expected register_file.txt. All 32 register values match the expected values, confirming correct handling of all forwarding, stalling, and flushing scenarios.

Register	Value	Register	Value
x0	0x0000000000000000	x16	0x0000000000000000
x1	0x000000000000000f	x17	0x0000000000000000
x2	0xfffffffffffffffb	x18	0x0000000000000000
x3	0x000000000000000a	x19	0x0000000000000000
x4	0x000000000000000a	x20	0x0000000000000000
x5	0x000000000000000a	x21	0x0000000000000000
x6	0xfffffffffffffffb	x22	0x0000000000000000
x7	0xfffffffffffffffb	x23	0x0000000000000000
x8	0x0000000000000000	x24	0x0000000000000000
x9	0x0000000000000000	x25	0x0000000000000000
x10	0x000000000000000f	x26	0x0000000000000000
x11	0xfffffffffffffffb	x27	0x0000000000000000
x12	0x0000000000000000	x28	0x0000000000000000
x13	0x000000000000001e	x29	0x0000000000000000
x14	0x0000000000000000	x30	0x0000000000000000
x15	0x0000000000000000	x31	0x0000000000000000

8.2 Cycle Count

Total clock cycles: **20**. The expected answer file reports 21 cycles (assuming branch resolution in the EX stage with a 2-cycle flush penalty). Our processor resolves branches in the ID stage, saving 1 cycle on the taken `beq x4, x5, 8` branch, yielding 20 cycles

total. This includes 4 cycles of pipeline fill latency, 1 flush cycle for the taken branch, and the execution of 15 instruction slots (14 real instructions + 1 NOP slot from the flush).

8.3 Hazard Scenarios Verified

Scenario	Verified By
EX-to-EX forwarding	`and x4` → `and x5` (ForwardB = 10)
MEM-to-EX forwarding	`and x4` → `and x5` → `or x6`(ForwardB=01 for x4)
Simultaneous EX + MEM forwarding	`addi x2` → `addi x3` → `add x1` (ForwardA = 01, ForwardB = 10)
Write-before-read bypass	`addi x2` → `addi x3` → `add x1` → `sub x2` (x2 via WB/ID bypass)
Branch taken + flush	`beq x4, x5, 8` taken; `beq x0, x0, 0` flushed with NOP
No-dependency branch	`beq x4, x5` operands written many cycles earlier; no forwarding or stall needed
Store + Load round-trip	`sd x1, 0(x5)` → `ld x10, 0(x5)`: x10 = 15 confirms correct memory write/read

9 BONUS sections

Beyond basic hazard handling, the design includes specific optimizations to improve cycle efficiency:

- **Store-Data Bypassing:** A specialized forwarding path (`Forward_sd`) was implemented to handle dependencies where a store instruction's data is produced by an instruction in the Write-Back stage. This avoids a 1-cycle stall in load-store sequences.
- **Early Evaluation:** By resolving branches in the ID stage, the stall penalty is halved compared to EX-stage resolution. This involves integrated stalling logic that accounts for dependencies $N-1$ and $N-2$ level instructions.
- **Integrated Hazard Logic:** The Hazard Detection Unit applies the minimum necessary stall cycles for each specific data dependency, and implements logic for branch as well as non-branch related Hazards.

10 Files specific to Pipelined Implementation

File	Purpose
`ForwardingUnit.v`	Data forwarding (EX/MEM hazard detection)
`HazardDetectionUnit.v`	Load-use and branch stall detection
`BranchForwardingUnit.v`	Branch comparator forwarding
`equalityComparator.v`	ID-stage branch condition evaluation
`mux4x2.v`	4-to-1 MUX for forwarding paths
`pipe_tb.v`	System-level testbench

11 Team Contributions

Member	Contribution
Yogesh Krishna	[Pipeline Registers, HazardDetectionUnit, BranchForwardingUnit, integration]
Pidapa Haarthiv	[BranchForwardingUnit, Custom Forward Logic units, Testing of codebase]
Shruti Dhasmana	[Report Making, Sequential Implementation - mapping to pipelining]